P R O J E C T D O C U M E N T A T I O N

ParseViz

Interactive LL(1) Grammar Practice Studio

Item	Details
Problem code	PS-03
Project type	Hackathon project
Subject	Compiler Design
Unit	Unit II: Parsing (Top-Down)
Topics	CFG, LL(1), ambiguity, left recursion, left factoring, FIRST/FOLLOW
Difficulty	Medium
Technology	HTML, CSS and JavaScript (single file, runs offline)
Team	Team 09
Submitted to	Uma Shankari
Submitted by	B.Chetna (2420030098) Vasavi (2420030785) Aashna (2420030647) Asleshitha (2420030740)
Section	9
GitHub repository	https://github.com/2420030785-vasavi/CD_-TEAM-09-PS-03-ParseViz---Interactive-Parse-Tree-Generator
Date	30 October 2026

Contents

1. Abstract

2. Introduction

3. Problem statement and objectives

4. Theoretical background

5. Proposed system

6. Algorithms and implementation

7. User guide and running the project

8. Testing and results

9. Mapping to the judging criteria

10. Limitations and future scope

11. Conclusion

12. References

13. Output screens

1. Abstract

ParseViz is an interactive LL(1) grammar practice studio built during a hackathon for problem statement PS-03 of the Compiler Design course (Unit II: Parsing, Top-Down). Students often memorise the procedures for removing left recursion and left factoring without understanding why they are needed. ParseViz lets a student type any context-free grammar and a test string, and then shows each step of the work: the grammar transformations, the FIRST and FOLLOW sets with the production behind every member, the LL(1) parsing table with conflicts highlighted, a step-by-step parse trace, and an animated parse tree or the exact error position. A backtracking parser is included for comparison, and a practice mode asks the student to predict whether strings are accepted or rejected. The whole system runs in a web browser from a single HTML file, with no installation.

2. Introduction

Context-free grammars, derivations and parse trees are among the most abstract topics in compiler design. Students often memorise the steps for removing left recursion and left factoring without understanding why those steps are needed. A tool that shows each transformation as it happens can bridge that gap.

ParseViz is an interactive grammar practice studio. A student enters a context-free grammar and a test string. The studio then shows the grammar transformations step by step, computes FIRST and FOLLOW sets with the reason for every member, builds the LL(1) parsing table, traces the parse with the stack, remaining input and action at every step, and draws the parse tree. A backtracking parser and a practice mode complete the tool.

3. Problem statement and objectives

The problem statement (PS-03) asks for a grammar practice studio in which a student can do the following. The table shows where each requirement is met.

No.	Requirement	Where it is implemented in ParseViz
1	Enter a grammar and choose a valid, invalid or epsilon-related test string	Sidebar: grammar editor, Valid / Invalid / Epsilon sample buttons, free-text string box, random valid string
2	Identify left recursion and show each removal step	Tab 1: Left recursion (immediate and indirect)
3	Find common prefixes and show the left-factored grammar	Tab 2: Left factoring
4	Compute FIRST and FOLLOW with contributing productions	Tab 3: FIRST / FOLLOW, click a member for its reason
5	Generate the LL(1) table, highlight empty entries, report conflicts	Tab 4: LL(1) table with conflict banner
6	Trace parsing with stack, input and action at every step	Tab 5: Parse & tree, trace table
7	Show the parse tree, or the error location on rejection	Tab 5: animated tree, highlighted error node and message

No.	Requirement	Where it is implemented in ParseViz
Bonus	Practice mode that checks accept/reject predictions	Tab 7: five practice drills with scoring
Extra	Backtracking parser for comparison	Tab 6: Backtracking

4. Theoretical background

This section recalls the concepts that ParseViz visualises, so that the rest of the document can be read on its own.

4.1 Context-free grammar

A context-free grammar is a 4-tuple G = (N, T, P, S): a set of nonterminals N, a set of terminals T, a set of productions P of the form A → α, and a start symbol S. The running example in this document is the classic expression grammar:

```
E -> E + T | T
T -> T * F | F
F -> ( E ) | id
```

4.2 Derivations and parse trees

A derivation starts from the start symbol and repeatedly replaces a nonterminal using one of its productions until only terminals remain. In a leftmost derivation the leftmost nonterminal is always replaced. For id + id the derivation is E ⇒ E + T ⇒ T + T ⇒ F + T ⇒ id + T ⇒ id + F ⇒ id + id. A parse tree shows the same derivation as a tree: the root is the start symbol, interior nodes are nonterminals, and the leaves, read from left to right, spell the string.

4.3 Ambiguity

A grammar is ambiguous when some string has more than one parse tree. The dangling-else grammar S → i E t S | i E t S e S | a is the standard example: in i b t i b t a e a, the else can belong to either if. A predictive parser needs one clear choice at every step, so ambiguity shows up as a conflict in the LL(1) table.

4.4 Left recursion

A grammar is left recursive if a nonterminal can derive a string that begins with itself, as in E → E + T. A top-down parser that expands E must first expand E again, without consuming any input, and so loops forever. The recursion is removed by rewriting the rule so that the nonterminal is followed, not preceded, by its repeating part.

4.5 Left factoring

For S → a b c | a b d, a parser that has read a cannot tell which alternative to use. Left factoring moves the shared prefix into one production and postpones the decision to a new nonterminal, so that one token of look-ahead is enough.

4.6 FIRST and FOLLOW

FIRST(α) is the set of terminals that can begin a string derived from α (with ε if α can vanish). FOLLOW(A) is the set of terminals that can appear immediately after A in some derivation, with $ for the end of input. These two sets decide where each production is placed in the parsing table.

4.7 LL(1) grammars

LL(1) means: scan the input Left to right, build a Leftmost derivation, and use 1 token of look-ahead. A grammar is LL(1) when, for every nonterminal A with two alternatives α and β, the following hold:

- FIRST(α) and FIRST(β) are disjoint.
- At most one of α and β can derive ε.
- If β can derive ε, then FIRST(α) and FOLLOW(A) are disjoint.

Equivalently, no cell of the LL(1) table holds more than one production.

4.8 Predictive parsing and backtracking

A predictive parser uses the table and a stack, never undoes a decision, and runs in time linear in the input. A backtracking parser has no table: it tries the alternatives of a nonterminal in order and returns to an earlier choice when a branch fails. It accepts a wider range of grammars but can take exponential time, and it cannot handle left recursion at all. ParseViz implements both, so the cost of guessing can be compared with the cost of looking ahead.

5. Proposed system

5.1 Scope and features

- Works for any context-free grammar typed in the editor; nothing is hard-coded.
- Six sample grammars: expression (left recursion), dangling else (conflict), epsilon productions, left factoring, indirect left recursion, balanced parentheses.
- Live analysis: results update while the grammar or the test string is typed.
- Dashboard cards at the top summarise nonterminals, terminals, left recursion steps, left factoring steps, LL(1) status, conflicts, input result and backtracks. Clicking a card opens the related tab.
- Runs fully offline in a browser from one HTML file; no server, installation or internet connection is needed.

5.2 System architecture

ParseViz is a single-page application. All computation happens in the browser in JavaScript. The analysis is a pipeline in which each stage uses the output of the previous one:

```
grammar text
  → parseG: read the productions into a grammar map
  → elimLR: remove left recursion (with step log)
  → leftFactor: factor common prefixes (with step log)
  → firstFollow: FIRST and FOLLOW with reasons
  → buildTable: LL(1) table and conflict list
  → runParse: predictive parse, trace and tree nodes
  → drawTree: SVG parse tree with step animation
```

Side modules: backtrack (Tab 6) and the practice drills (Tab 7)

5.3 Modules

Function	Purpose	Shown in
parseG	Parses the editor text; reports bad lines and undefined nonterminals	Sidebar errors
elimLR	Immediate and indirect left recursion removal, with step log	Tab 1
leftFactor	Repeated common-prefix factoring, with step log	Tab 2
firstFollow	Fixed-point FIRST and FOLLOW; stores the reason for each member	Tab 3
buildTable	Fills M[A, t]; lists cells with more than one production	Tab 4
runParse	Stack-based predictive parser; builds trace rows and tree nodes	Tab 5
drawTree	Lays out and draws the tree; reveals nodes by step	Tab 5
backtrack	Recursive-descent parser that tries alternatives and undoes failures	Tab 6
qAR, qLR, qTL, qFF, qNA	Question generators for the five practice drills	Tab 7

5.4 Data structures

- Grammar: a map from each nonterminal to a list of productions; a production is a list of symbols, and an empty list means ε.
- FIRST and FOLLOW: one set per nonterminal, plus a map recording which production first added each member.
- Parsing table: M[A][t] holds a list of candidate productions with the reason for each; more than one entry is a conflict.
- Parse tree node: id, symbol, children, a terminal flag and the step at which it was matched.
- Trace row: stack text, remaining input, action text and the number of tree nodes created so far, which lets the animation replay the tree.

5.5 Technology

HTML and CSS for the interface, plain JavaScript for every algorithm, and SVG for the parse tree. No external library is required, so the project opens directly from the file system.

6. Algorithms and implementation

6.1 Grammar input

Convention	Example
One rule per line, "->" or "→"	E -> E + T | T
Alternatives separated by |	F -> (E) | id
Symbols separated by spaces	T -> T * F | F
Nonterminal: starts with a capital letter	E, T, F, Expr
Any other symbol is a terminal; a word such as id is one token	id, +, (,)
Epsilon written as ε, eps or an empty alternative	A -> a A | eps
The first rule's left side is the start symbol	E

6.2 Left recursion elimination

A grammar is left recursive when a nonterminal can derive a string that starts with itself. A top-down parser expands such a nonterminal forever, so the recursion must be removed. ParseViz handles immediate recursion directly and indirect recursion by substitution.

```
order the nonterminals A1 ... An
for i = 1 to n:
   for j = 1 to i-1:
      replace each Ai -> Aj g by Ai -> d1 g | d2 g | ...
                              where Aj -> d1 | d2 | ...
   split the alternatives of Ai into
         recursive: Ai -> Ai a1 | Ai a2 | ...
         others: Ai -> b1 | b2 | ...
   if there are recursive alternatives:
         Ai -> b1 Ai' | b2 Ai' | ...
         Ai' -> a1 Ai' | a2 Ai' | ... | epsilon
```

Worked example. For E → E + T | T, the recursive tail is α = + T and the other alternative is β = T. The rewrite is E → T E′ and E′ → + T E′ | ε. T → T * F | F is handled the same way. The grammar after this step is:

```
E → T E'
T → F T'
F → ( E ) | id
E' → + T E' | ε
T' → * F T' | ε
```

The ε alternative is essential: it lets the repetition stop. Placing E′ after β makes the first symbol of every alternative a symbol other than E, which removes the loop.

6.3 Left factoring

When two alternatives of a nonterminal start with the same symbols, a predictive parser cannot choose between them with one token of look-ahead. Left factoring pulls the common prefix out into a new nonterminal.

```
repeat
   for each nonterminal A:
      group the alternatives of A by their first symbol
      if some group has two or more alternatives:
         a = longest common prefix of the group
         create a new nonterminal A'
         A -> a A' | (alternatives not in the group)
         A' -> the suffixes that follow a
until no group is found
```

Worked example (the "Left factoring" sample): S → a b c | a b d | a e | f. The common prefix a gives S → f | a S′ and S′ → b c | b d | e. The alternatives of S′ still share b, so a second pass gives S′ → e | b S″ and S″ → c | d. The final grammar is:

```
S → f | a S'
S' → e | b S"
S" → c | d
```

The expression grammar has no shared prefixes, so the tool reports that there is nothing to factor.

6.4 FIRST and FOLLOW sets

FIRST(X) is the set of terminals that can begin a string derived from X, with ε included when X can derive the empty string. FOLLOW(A) is the set of terminals that can appear immediately after A in some sentential form, with $ marking the end of input.

- FIRST of a terminal is the terminal itself.
- For A → X1 X2 ... Xn, add FIRST(X1) without ε to FIRST(A); continue to X2 only if X1 can derive ε, and so on. If every Xi can derive ε, add ε.
- FOLLOW(start symbol) contains $.
- For A → α B β, add FIRST(β) without ε to FOLLOW(B).
- If β is empty or can derive ε, add FOLLOW(A) to FOLLOW(B).
- Repeat until no set changes (fixed-point iteration).

Each time a member is added for the first time, ParseViz stores the production responsible. Clicking a member in Tab 3 shows that reason, for example "id ∈ FIRST(E) because of E → T E‴". For the expression grammar after transformation the sets are:

Nonterminal	FIRST	FOLLOW
E	{ (, id }	{ $,) }
T	{ (, id }	{ +, $,) }
F	{ (, id }	{ *, +, $,) }
E′	{ +, ε }	{ $,) }

Nonterminal	FIRST	FOLLOW
T′	{ *, ε }	{ +, $,) }

6.5 LL(1) table construction and conflict detection

```
for each production A -> a:
   for each terminal t in FIRST(a): put A -> a in M[A, t]
   if a can derive epsilon:
      for each terminal t in FOLLOW(A) (incl. $): put A -> a in M[A, t]

a cell that holds two or more productions is a conflict
   => the grammar is not LL(1)
```

Cells holding one production are shaded, empty cells show a dot (a syntax error if reached) and conflict cells are highlighted, with a banner that lists every conflict. The table for the expression grammar is:

	(	)	*	+	id	$
E	E → T E′	-	-	-	E → T E′	-
T	T → F T′	-	-	-	T → F T′	-
F	F → (E)	-	-	-	F → id	-
E′	-	E′ → ε	-	E′ → + T E′	-	E′ → ε
T′	-	T′ → ε	T′ → * F T′	T′ → ε	-	T′ → ε

Conflict example. The dangling-else grammar S → i E t S | i E t S e S | a, E → b is ambiguous. After factoring, the table reports:

```
M[S', e]: S' → ε and S' → e S
```

When a conflicting cell is reached during parsing, ParseViz prefers the non-ε production, which matches each else with the nearest if.

6.6 Predictive parsing with a trace

```
push $ and the start symbol; i = 0
loop:
   X = top of stack, a = input[i]
   if X is $: accept if a is $, else error
   if X is terminal: if X == a, pop and i++ ; else error
   if X nonterminal: if M[X, a] is empty: error
                                  else pop X, push the right side in reverse,
                                        create tree nodes for its symbols
```

Every action adds a row to the trace. For the input id + id * id on the transformed expression grammar, the parser accepts in 17 steps:

Step	Stack (top first)	Remaining input	Action
1	T E′ $	id + id * id $	Output E → T E′
2	F T′ E′ $	id + id * id $	Output T → F T′
3	id T′ E′ $	id + id * id $	Output F → id
4	id T′ E′ $	id + id * id $	Match "id"
5	E′ $	+ id * id $	Output T′ → ε
6	+ T E′ $	+ id * id $	Output E′ → + T E′
7	+ T E′ $	+ id * id $	Match "+"
8	F T′ E′ $	id * id $	Output T → F T′
9	id T′ E′ $	id * id $	Output F → id
10	id T′ E′ $	id * id $	Match "id"
11	* F T′ E′ $	* id $	Output T′ → * F T′
12	* F T′ E′ $	* id $	Match "*"
13	id T′ E′ $	id $	Output F → id
14	id T′ E′ $	id $	Match "id"
15	E′ $	$	Output T′ → ε
16	$	$	Output E′ → ε
17	$	$	Accept

Error handling. For id + * id the parser stops at the third token. The tool reports: At token 3 "*": T cannot start with this. Expected one of: (, id. The same message names the failing token, highlights it in the sidebar, and marks the stuck nonterminal with a highlighted box in the partial tree.

6.7 Parse tree generation and layout

- Each expansion creates child nodes under the nonterminal on top of the stack; ε productions create an ε leaf.
- Layout: leaves receive consecutive horizontal positions from left to right; every parent is centred over its first and last child; vertical position is the depth.
- Animation: each trace row records how many nodes exist after that step. Play, step and slider controls reveal nodes up to that count, so the tree grows exactly as the parser works. Terminals are highlighted once matched.
- For a rejected string the tree built so far is kept and the failing node is drawn in a highlighted box.

6.8 Backtracking parser

The backtracking tab shows how a top-down parser works without a table. It expands the leftmost nonterminal, tries each alternative in order, matches terminals against the input, and undoes its choice when a branch fails.

```
go(form, pos):
  if form is empty: return pos == n
  X = first symbol of form
```

```
if X is terminal: if input[pos] == X: match, go(rest, pos+1) ; else fail
else for each alternative of X (in order):
        log "try X -> alt"
        if go(alt + rest, pos): return true
        log "backtrack"
    return false
```

A guard stops the search when the same nonterminal is expanded more than six times at the same input position, which is the signature of left recursion; the tab then explains why it stopped. A step limit protects the browser from very long searches. For a successful parse the tab prints the leftmost derivation that was found.

Comparison for id + id * id on the expression grammar: the LL(1) parser needs 17 steps and never undoes anything. The backtracking parser on the left-recursion-free grammar logs 34 steps, tries 17 alternatives and backtracks 6 times. On the original grammar it cannot terminate and the guard reports left recursion.

Grammar	Input	LL(1) steps	Backtracking steps	Backtracks
Expression	id + id * id	17	34	6
Dangling else	i b t a e a	12	43	9
Epsilon productions	a a b	10	15	2
Left factoring	a b d	7	11	1
Indirect left recursion	y x x	9	11	1
Balanced parentheses	(( )) ( )	14	25	4

6.9 Practice mode

Practice mode has five drills, each played in rounds of ten questions with a difficulty setting. Every correct answer is computed by the real algorithms, so there is no stored answer key.

Drill	Question	Points
Accept or reject	Will the LL(1) parser accept this string? After a correct reject, click the token where it fails.	10 (+5 bonus)
Remove left recursion	Choose the correct rewrite of a left-recursive rule from four options; the wrong options are common mistakes.	12
Table lookup	Which production is in M[A, t]?	10
FIRST / FOLLOW	Select every member of FIRST(A) or FOLLOW(A).	12
Next action	Given the stack and remaining input, what does the parser do next?	8

- Valid strings are produced by a random derivation of the grammar; once a derivation is deep, the alternative with the fewest nonterminals is chosen so it always ends.
- Invalid strings are made by deleting, inserting, replacing or swapping a token in a valid string, and are checked with the parser; the parser decides the correct answer.

- Easy uses strings of up to 6 tokens, Medium up to 10 and Hard up to 16 tokens, and Hard also asks about the new nonterminals such as E′.
- A streak of three correct answers adds 2 bonus points per answer. The panel shows score, accuracy, streak, best streak, per-drill results and a progress bar for the round.
- A reference box above every question shows the student's grammar and the transformed grammar the table is built from.

7. User guide and running the project

1. Open index.html in a modern browser such as Chrome or Edge.
2. Choose an example grammar, or type your own productions in the editor.
3. Type a test string, or press Valid sample, Invalid sample, Epsilon sample or Random valid string. Results appear as you type.
4. Read the sidebar result, then move through the tabs: Left recursion, Left factoring, FIRST / FOLLOW, LL(1) table, Parse & tree, Backtracking.
5. In Parse & tree, press Play or use the step buttons and slider; click a trace row to jump to it.
6. Open Practice, pick a drill and a difficulty, and answer ten questions.

7.1 Repository contents

```
ParseViz/
  index.html (the complete application)
  README.md (project overview for GitHub)
  screenshots/ (output screens, PNG)
  ParseViz_Project_Documentation.docx (this document)
  ParseViz_Presentation.pptx (project presentation)
```

7.2 Running the project from GitHub

1. Open the repository at https://github.com/2420030785-vasavi/CD_-TEAM-09-PS-03-ParseViz---Interactive-Parse-Tree-Generator and choose Code, then either clone it with git or download the ZIP file and extract it.
2. Open index.html in a modern browser such as Chrome or Edge. Nothing needs to be installed, and no internet connection is required.
3. Optional: to host the project online, open the repository Settings, choose Pages, select the main branch and the root folder, and save. GitHub then publishes index.html at a public address that can be shared with the evaluator.

8. Testing and results

Each sample grammar was run with a valid, an invalid and an epsilon-related string. The table shows the outcome of the LL(1) parser together with the reason reported for rejected strings.

Grammar	Input	Result	Reason for rejection
Expression	id + id * id	Accepted	-

Grammar	Input	Result	Reason for rejection
	id + * id	Rejected	At token 3 "*": T cannot start with this. Expected one of: (, id.
	(empty string)	Rejected	At end of input: E cannot start with this. Expected one of: (, id.
Dangling else	i b t a e a	Accepted	-
	i b t i b t a e a	Accepted	-
	i b t	Rejected	At end of input: S cannot start with this. Expected one of: a, i.
Epsilon productions	a a b	Accepted	-
	b a	Rejected	At token 2 "a": B cannot start with this. Expected one of: b, $.
	(empty string)	Accepted	-
Left factoring	a b d	Accepted	-
	a b	Rejected	At end of input: S″ cannot start with this. Expected one of: c, d.
	f	Accepted	-
Indirect left recursion	y x x	Accepted	-
	x y	Rejected	At token 1 "x": S cannot start with this. Expected one of: y.
	(empty string)	Rejected	At end of input: S cannot start with this. Expected one of: y.
Balanced parentheses	(( )) ( )	Accepted	-
	(( )	Rejected	Token 4 is "end of input" but ")" was expected.
	(empty string)	Accepted	-

8.1 Edge cases

- **Epsilon productions.** S → A B, A → a A | ε, B → b B | ε accepts the empty string; E′ → ε entries appear in the FOLLOW-based cells of the table.
- **Empty input.** The test box may be empty; the parser then works on $ alone and accepts only if the start symbol can derive ε.
- **Indirect left recursion.** S → A, A → S x | y is rewritten into A → y A′, A′ → x A′ | ε, and the result is LL(1).
- **Multiple conflicts.** Every conflicting cell is listed in the banner, not only the first.
- **Unknown tokens.** A token that is not a terminal of the grammar is highlighted and causes an error at that position.
- **Left recursion in backtracking.** The loop guard stops the search and names left recursion as the cause.

8.2 Verification

The algorithms were checked by running every sample grammar through the full pipeline and comparing the transformed grammars, FIRST and FOLLOW sets, tables and parse results with hand-worked answers. The practice generators were run repeatedly on every sample grammar at all three difficulty levels without errors.

9. Mapping to the judging criteria

Criterion	How it is addressed
Correctness of left recursion elimination	Standard algorithm with substitution for indirect cases; every step is logged with α, β and the new rules.
Correctness of FIRST and FOLLOW	Fixed-point iteration; the reason for every member is stored and can be inspected.
LL(1) table and conflict detection	Table built from FIRST and FOLLOW; all multi-entry cells are reported, not just the first.
Parse tree quality and animation	Centred tree layout, step-by-step reveal with play, pause, speed and slider; matched terminals and the error node are highlighted.
Edge cases	ε productions, empty input, indirect recursion, several conflicts and unknown tokens are handled.
Bonus: practice mode	Five drills with generated questions, answer checking by the real parser, scoring, streaks and rounds.

10. Limitations and future scope

Limitations

- Ambiguous grammars are reported as non-LL(1); the tool does not rewrite them into unambiguous grammars.
- In a conflicting cell the parser must choose one production; ParseViz prefers the non-ε one, which is only one possible policy.
- The backtracking parser can take exponential time on some grammars, so a step limit of 3000 logged steps is used.
- Grammar symbols are separated by spaces; a multi-letter word is read as one token.

Future scope

- Bottom-up parsing: LR(0), SLR(1) and LALR(1) item sets and tables.
- Export of the tree, tables and step logs as images or a PDF report.
- Saving and sharing grammars through a link.
- Error recovery strategies such as panic mode, shown on the trace.

11. Conclusion

ParseViz turns the abstract steps of top-down parsing into something a student can see and test. Left recursion removal, left factoring, FIRST and FOLLOW, the LL(1) table, the parse trace, the parse tree

and a backtracking parser are all computed live for any grammar, and the practice mode lets students check their own predictions against the real parser. The project meets every requirement of problem statement PS-03 and the bonus challenge.

12. References

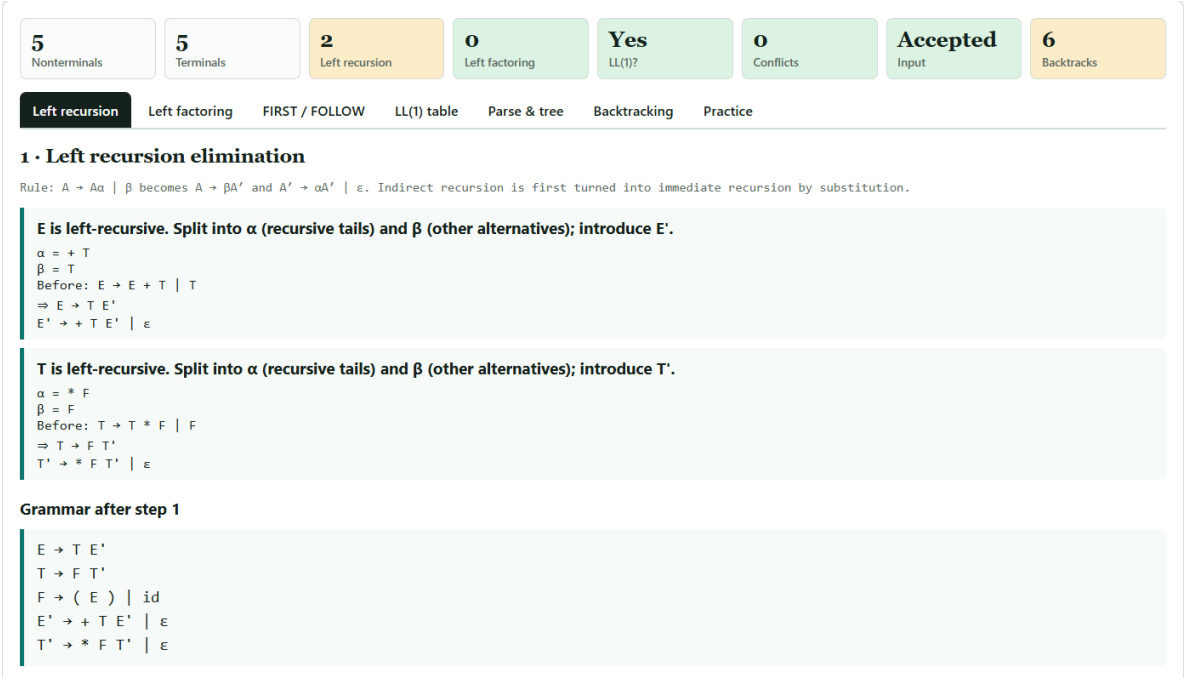
- Aho, A. V., Lam, M. S., Sethi, R. and Ullman, J. D. Compilers: Principles, Techniques, and Tools, 2nd ed. Pearson, 2006.
- Cooper, K. D. and Torczon, L. Engineering a Compiler, 2nd ed. Morgan Kaufmann, 2011.
- Grune, D. and Jacobs, C. J. H. Parsing Techniques: A Practical Guide, 2nd ed. Springer, 2008.

13. Output screens

The following screens are taken from the running application. All of them use the expression grammar E → E + T | T, T → T * F | F, F → (E) | id and the test string id + id * id. The dashboard cards at the top of every screen show the live summary: 5 nonterminals, 5 terminals, 2 left-recursion steps, 0 left-factoring steps, LL(1) = Yes, 0 conflicts, input Accepted and 6 backtracks.

Output 1: Left recursion elimination

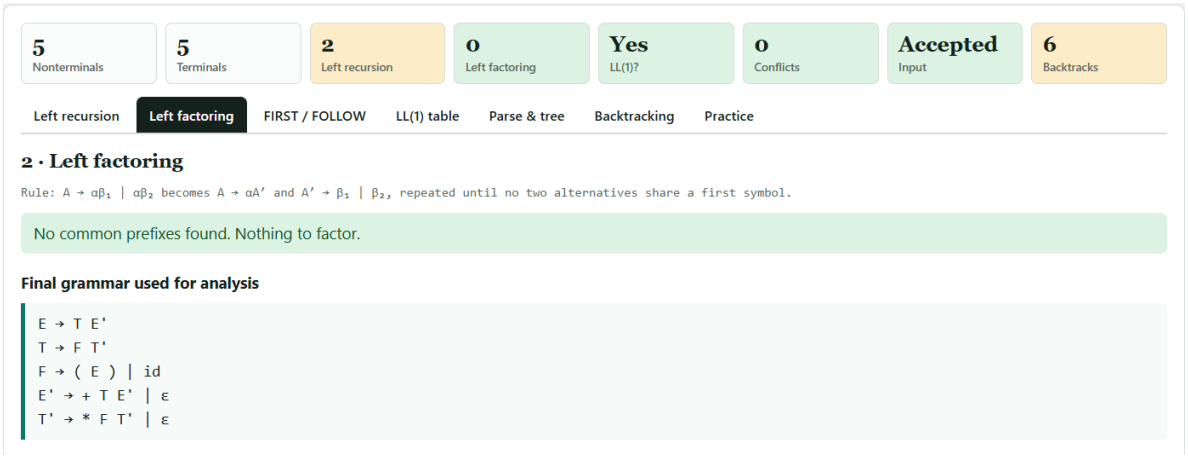
What it shows. E and T are both left recursive. For each one the tab lists α (the recursive tail), β (the other alternatives), the original rule and the rewritten rules with the new nonterminals E′ and T′. Below, the grammar after this step is printed in full.



Output 1: Left recursion elimination

Output 2: Left factoring

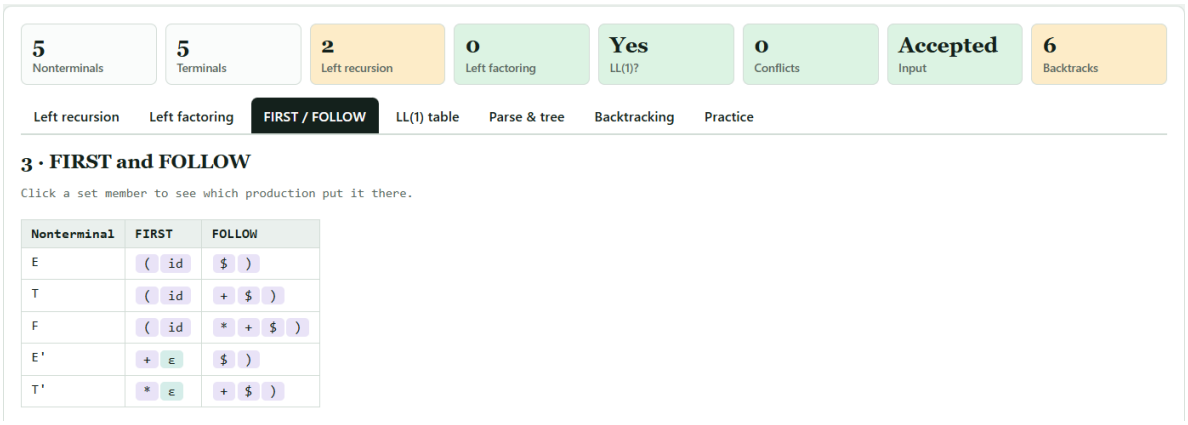
What it shows. The tab applies the common-prefix rule and reports that this grammar has no alternatives with a shared first symbol, so nothing needs to be factored. The final grammar used for all later analysis is displayed.



Output 2: Left factoring

Output 3: FIRST and FOLLOW sets

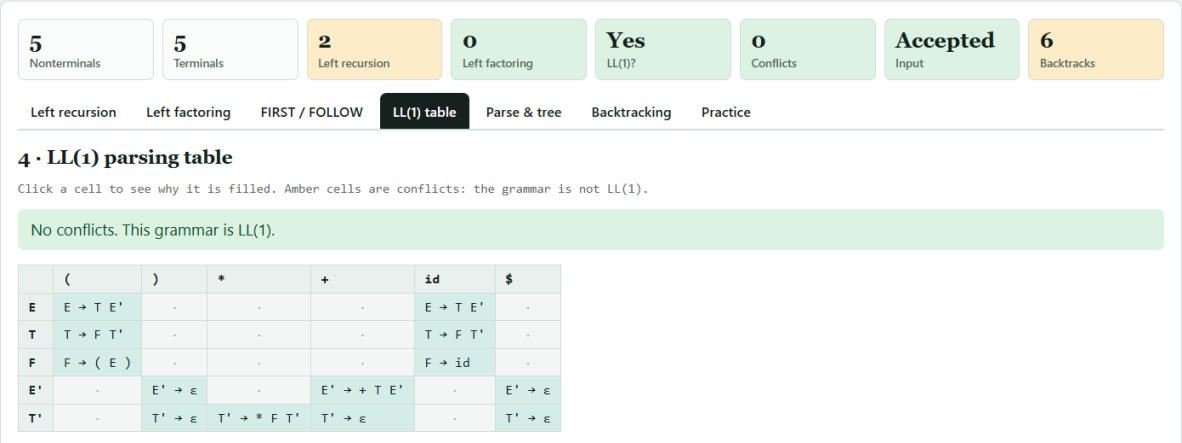
What it shows. FIRST and FOLLOW for every nonterminal of the transformed grammar, for example FIRST(E) = { (, id } and FOLLOW(E) = { $,) }. Each member can be clicked to see the production that put it there.



Output 3: FIRST and FOLLOW sets

Output 4: LL(1) parsing table

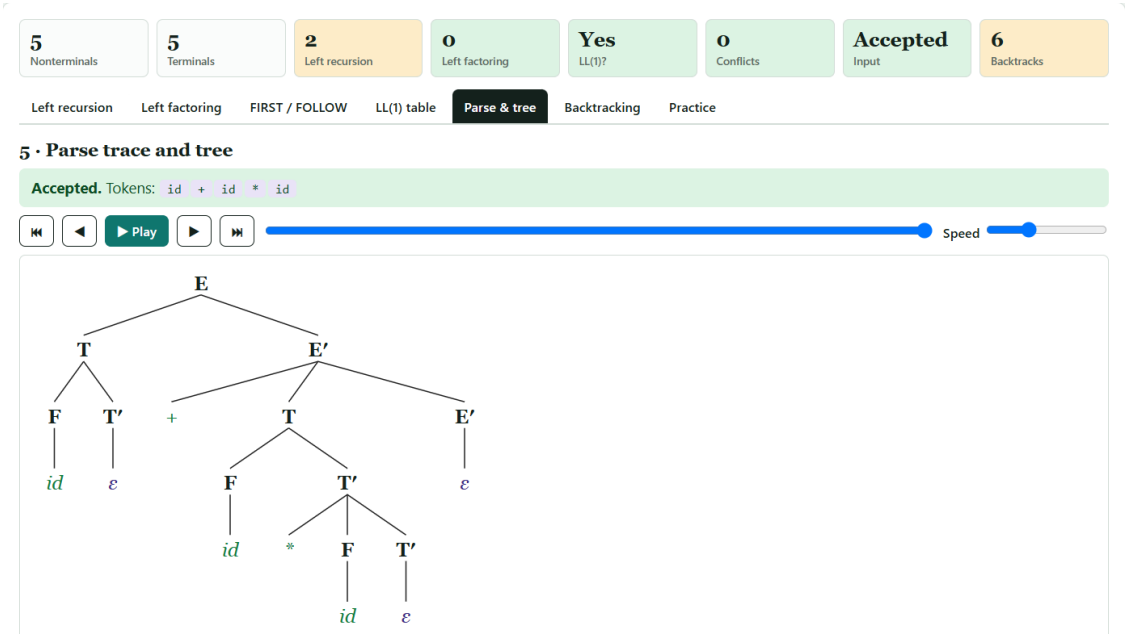
What it shows. The table M[A, t] with productions in shaded cells and empty cells shown as dots. The banner above the table confirms there are no conflicts, so the grammar is LL(1). Entries such as E′ → ε appear under the FOLLOW symbols) and $.



Output 4: LL(1) parsing table

Output 5: Parse trace and parse tree

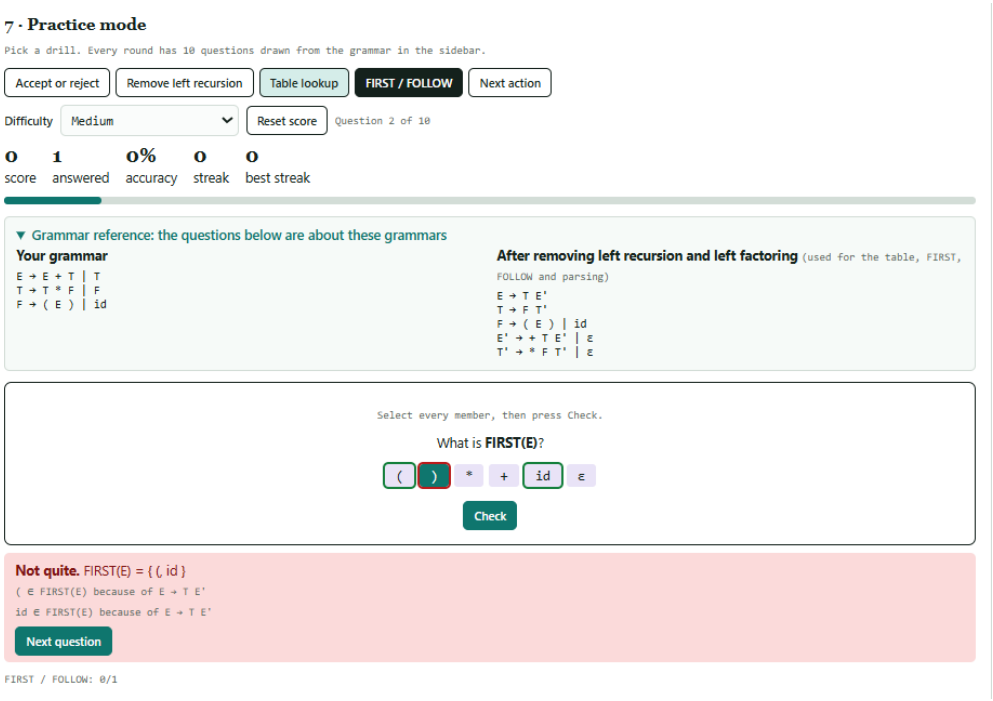
What it shows. The accepted input id + id * id with its tokens, the playback controls (first, previous, play, next, last, speed) and the parse tree built so far. Nonterminals are plain letters, terminals are italic, and ε leaves show where a production vanished.



Output 5: Parse trace and parse tree

Output 6: Practice mode

What it shows. A FIRST / FOLLOW drill. The grammar reference box shows the student's grammar and the transformed grammar. The student selects the members of FIRST(E); the screen then marks the right and wrong choices and explains each member. Score, accuracy and streak appear above.



Output 6: Practice mode